

Six_XX_GRUFF_TTR

March 23, 2025

1 Comparative Analysis of Literary Translations in the Odyssey: Lexical Diversity and Frequency Trends

1.1 Research Objectives

1.2 Methodology Roadmap (Table of Contents)

1.2.1 1 Libraries & Text Acquisition

Setting up the environment and loading texts.

1.2.2 2 Experiment 1: Type-Token Ratio (TTR)

- **Implementation:** Computing TTR for both translations.
- **Parametric Hypothesis Testing:** Comparing TTR statistically.

```
[1]: ##### Pandas set-up
import numpy as np
import pandas as pd

from tqdm.notebook import tqdm
import re
tqdm.pandas()

pd.set_option("display.max_colwidth", None) # Prevent truncation of long values
pd.set_option("display.max_rows", None) # Show all rows
pd.set_option("display.max_columns", None) # Show all columns
pd.set_option("display.expand_frame_repr", False) # Prevent wrapping in
↳DataFrames
```

```
[2]: ##### Other libraries
import re
import nltk

from collections import Counter

import sys
import os
```

```
[3]: ##### Visualization
%matplotlib inline
sys.path.append('/Users/debr/English-Homer')
import matplotlib.pyplot as plt
import seaborn as sns
import bard_visualization as viz # My Vizualization library
```

```
[4]: # Import my functions
sys.path.append('/Users/debr/English-Homer/functions')
import e_nlp as e
```

Functions for NLP are live! use e.<function> to call them.

Downloading NLTK resources...

Download complete.

[nltk_data] Downloading package punkt_tab to /Users/debr/nltk_data...

[nltk_data] Package punkt_tab is already up-to-date!

```
[5]: ##### Cell display options
pd.set_option("display.max_colwidth", None) # Prevent truncation of long values
pd.set_option("display.max_rows", None) # Show all rows
pd.set_option("display.max_columns", None) # Show all columns
pd.set_option("display.expand_frame_repr", False) # Prevent wrapping in
↳DataFrames
```

```
[6]: ##### File management
# TO UPDATE
nb_id = "Six_XXth_TTR"
odysseys = ["AT_Murray", "Fitzgerald", "Lattimore", "Fagles", "Wilson", "Green"]

# Paths
output_path = f"/Users/debr/English-Homer/Six_XXth_Lexical/{nb_id}/"
os.makedirs(os.path.dirname(output_path), exist_ok=True)
output_path_plots = f"/Users/debr/English-Homer/Six_XXth_Lexical/{nb_id}_plots/"
os.makedirs(os.path.dirname(output_path_plots), exist_ok=True)
```

```
[7]: # Daframe with all the odysseys
dfs = []

for odyssey in odysseys:
    filepath = f"/Users/debr/odysseys_en/Odyssey_dfs/Odyssey_{odyssey}_eda_END.
↳csv"
    temp_df = pd.read_csv(filepath)
    dfs.append(temp_df) # Append it to the list

df = pd.concat(dfs, axis=0, ignore_index=True)

df = df[['author', 'book_num', 'text', 'tokens', 'num_words', 'num_tokens']]
```

```
df["diff"] = df["num_words"] - df["num_tokens"]
e.check_df(df)
```

No missing values

```
df.columns: Index(['author', 'book_num', 'text', 'tokens', 'num_words',
                  'num_tokens',
                  'diff'],
                  dtype='object')
```

Shape: (144, 7)

1.3 Experiment 1: Type-Token Ratio (TTR)

1.4 Introduction

Type-Token Ratio (TTR) is a key measure of lexical diversity, calculated as the number of unique words (**types**) divided by the total word count (**tokens**). It provides insight into a translator's lexical choices when rendering the same source text.

- A **higher TTR** suggests a richer vocabulary, possibly reflecting an effort to capture nuances or stylistic complexity.
- A **lower TTR** may indicate a more repetitive or constrained word choice, potentially prioritizing accessibility or fidelity to the original.

1.4.1 TTR Formula

$$TTR = \left(\frac{\text{Unique Words}}{\text{Total Words}} \right) \times 100$$

```
[8]: import scipy.stats as stats

# Compute TTR
df["ttr"] = df["tokens"].apply(lambda x: (len(set(x)) / len(x) * 100) if x else 0)
```

```
[9]: # List of translators
translators = ["AT_Murray", "Fitzgerald", "Lattimore", "Fagles", "Wilson",
               "Green"]
ttr_by_translator = {}
for translator in translators:
    ttr_by_translator[translator] = df[df["author"] == translator]["ttr"].
    tolist()

# TTR DF by translator
if 'book_num' not in df.columns:
```

```

df['book_num'] = [f"Book_num_{i+1}" for i in range(1, 25)] * len(translators)
ttr_df = df.pivot(index='book_num', columns='author', values='ttr')

# Reorder columns if needed
ttr_df = ttr_df[translators]
e.check_df(ttr_df)

```

No missing values

```
df.columns: Index(['AT_Murray', 'Fitzgerald', 'Lattimore', 'Fagles', 'Wilson', 'Green'], dtype='object', name='author')
```

Shape: (24, 6)

```

[10]: from scipy import stats

for translator in translators:
    # Get the TTR data for this translator
    ttr_data = ttr_by_translator[translator]

    # Perform Shapiro-Wilk test
    stat, p_value = stats.shapiro(ttr_data)

    # Print results
    print(f"Shapiro-Wilk test for {translator}'s data: T-statistic={stat:.4f}, p-value={p_value:.4f}")

    # Interpret results
    if p_value < 0.05:
        print(f"{translator}'s TTR data is not normally distributed.")
    else:
        print(f"{translator}'s TTR data is normally distributed.")

    print() # Add empty line for readability

```

Shapiro-Wilk test for AT_Murray's data: T-statistic=0.9678, p-value=0.6131
AT_Murray's TTR data is normally distributed.

Shapiro-Wilk test for Fitzgerald's data: T-statistic=0.9608, p-value=0.4550
Fitzgerald's TTR data is normally distributed.

Shapiro-Wilk test for Lattimore's data: T-statistic=0.9702, p-value=0.6712
Lattimore's TTR data is normally distributed.

Shapiro-Wilk test for Fagles's data: T-statistic=0.9817, p-value=0.9245
Fagles's TTR data is normally distributed.

Shapiro-Wilk test for Wilson's data: T-statistic=0.9666, p-value=0.5843
Wilson's TTR data is normally distributed.

Shapiro-Wilk test for Green's data: T-statistic=0.9719, p-value=0.7134
Green's TTR data is normally distributed.

```
[11]: import scipy.stats as stats
import numpy as np
from itertools import combinations

# Use the ttr_by_translator dictionary
translator_names = translators # Use your existing list of translators
ttr_values = [ttr_by_translator[translator] for translator in translator_names]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(*ttr_values)
print(f"F-statistic: {f_stat:.4f}, P-value: {p_value:.4f}")

if p_value < 0.05:
    print("There are statistically significant differences in TTR among the_
    ↪ translators.")
else:
    print("There are no statistically significant differences in TTR among the_
    ↪ translators.")

# Perform pairwise t-tests with Bonferroni correction
print("\nPairwise comparisons:")

# Number of comparisons for Bonferroni correction
num_comparisons = len(list(combinations(range(len(translator_names)), 2)))

for i, j in combinations(range(len(translator_names)), 2):
    t_stat, p_val = stats.ttest_ind(ttr_values[i], ttr_values[j])

    # Apply Bonferroni correction
    adj_p_val = min(p_val * num_comparisons, 1.0)

    # Calculate mean difference
    mean_diff = np.mean(ttr_values[i]) - np.mean(ttr_values[j])

    # Determine significance
    is_significant = "Significant" if adj_p_val < 0.05 else "Not significant"

    print(f"{translator_names[i]} vs {translator_names[j]}: Diff = {mean_diff:.
    ↪ 4f}, p = {adj_p_val:.4f} - {is_significant}")
```

F-statistic: 9.2453, P-value: 0.0000

There are statistically significant differences in TTR among the translators.

Pairwise comparisons:

AT_Murray vs Fitzgerald: Diff = 0.0007, p = 1.0000 - Not significant
AT_Murray vs Lattimore: Diff = 0.0260, p = 0.0703 - Not significant
AT_Murray vs Fagles: Diff = 0.0240, p = 0.1341 - Not significant
AT_Murray vs Wilson: Diff = -0.0252, p = 0.2942 - Not significant
AT_Murray vs Green: Diff = 0.0224, p = 0.2160 - Not significant
Fitzgerald vs Lattimore: Diff = 0.0253, p = 0.1251 - Not significant
Fitzgerald vs Fagles: Diff = 0.0233, p = 0.2249 - Not significant
Fitzgerald vs Wilson: Diff = -0.0259, p = 0.3048 - Not significant
Fitzgerald vs Green: Diff = 0.0217, p = 0.3473 - Not significant
Lattimore vs Fagles: Diff = -0.0020, p = 1.0000 - Not significant
Lattimore vs Wilson: Diff = -0.0512, p = 0.0001 - Significant
Lattimore vs Green: Diff = -0.0036, p = 1.0000 - Not significant
Fagles vs Wilson: Diff = -0.0492, p = 0.0002 - Significant
Fagles vs Green: Diff = -0.0016, p = 1.0000 - Not significant
Wilson vs Green: Diff = 0.0476, p = 0.0003 - Significant

```
[13]: # Perform t-test
t_stat, p_value = stats.ttest_ind(ttr_by_translator['Wilson'],
    ↪ttr_by_translator['Green'])
print(f"T-statistic: {t_stat}, P-value: {p_value}")
if p_value < 0.05:
    print("The difference in TTR between Wilson and Green is statistically
    ↪significant.")
else:
    print("The difference in TTR between Wilson and Green is not statistically
    ↪significant.")
```

T-statistic: 4.784093462932383, P-value: 1.807252159047434e-05

The difference in TTR between Wilson and Green is statistically significant.

```
[ ]: 
[ ]: 
[ ]:
```